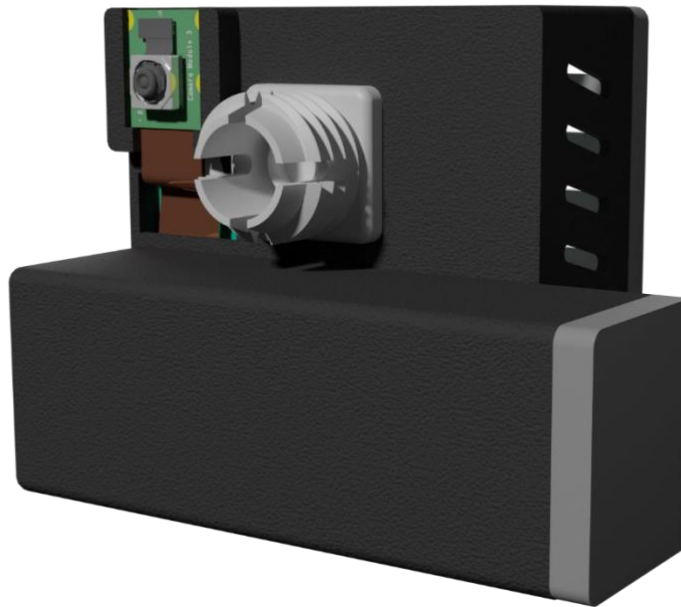




Gymnasiearbete

PiVision – mobilt förarstödssystem

Gustav Gamstedt & Liam Thorsén | Hitachigymnasiet Västerås |
Gymnasiearbete, Teknikprogrammet | Vårterminen 2025 |Handledare: Pär Henriksson |
Hitachigymnasiet Västerås | Extern handledare: Agustín Corbat från Institutet för
informationsteknologi på Uppsala universitet



Sammanfattning

Denna rapport presenterar PiVision, ett system utformat för att öka trafiksäkerheten i äldre fordon genom moderna teknologiska lösningar. Projektet har som mål att lösa problemet med att äldre fordon ofta saknar de vanliga säkerhetsfunktionerna som finns i nyare bilar, exempelvis kollisionssvarning och avståndsmätningssystem. Med hjälp av en Raspberry Pi försedd med en kamera och artificiell intelligens genom ett AI-kort kan PiVision analysera trafiksituationer i realtid, upptäcka närliggande fordon, beräkna avstånd och varna förare för potentiella kollisionrisker.

Systemet består av flera integrerade komponenter: en specialdesignad 3D-utskriven hållare monterad på vindrutan, en Raspberry Pi med ett AI-accelerationskort, en kamera och ett strömförsörjningssystem. Mjukvaran bearbetar kamerabilder genom AI för att identifiera fordon, uppskatta avstånd med matematiska beräkningar och fastställa kollisionrisk genom att analysera omkringliggande fordons bana. Ett webbaserat gränssnitt visar realtidsinformation till föraren på dess mobil, vilken bland annat inkluderar avstånd till närliggande fordon och varningssignaler.

Tester visade att systemet kan analysera 20 bilder per sekund, vilket är tillräckligt för kollisionssvarning i realtid. PiVision demonstrerade att det är möjligt att förbättra säkerheten hos äldre fordon på ett kostnadseffektivt och hållbart sätt med hjälp av modern teknik. Detta bidrar således till att minska behovet av att köpa en ny bil.

Abstract

PiVision is a system designed to enhance the safety of older vehicles by implementing modern safety features using artificial intelligence technology. The project addresses the safety disparity between new cars, which come equipped with advanced driver assistance systems, and older vehicles lacking these features. Using a Raspberry Pi equipped with a camera and AI capabilities, PiVision monitors surrounding traffic to detect potential collision risks.

The hardware components include a Raspberry Pi, an AI accelerator board, a camera, and a battery power system, all housed in a custom-designed 3D-printed holder that attaches to the windshield. The software utilizes Python with various libraries for image processing, deep learning-based detection, distance calculation, and web server functionality.

The system captures images through a camera mounted on the windshield, processes them using computer vision techniques, and raises an alarm when collision risk is high. Through custom algorithms, PiVision calculates both the forward and lateral distances to nearby vehicles, estimates collision risk in real-time, and provides appropriate warnings to the driver through a web-based interface displayed on a mobile device.

Testing demonstrated that the system can effectively operate at approximately 20 frames per second, providing sufficient performance for real-time collision warning. Furthermore, PiVision is a more cost-effective and sustainable solution, compared to purchasing a new car.

Innehåll

1. Inledning	4
1.1 Bakgrund	4
1.2 Syfte och frågeställningar	4
2. Metod	5
2.1 Planering	5
2.2 Programmering	5
2.3 Elektronik	5
2.4 CAD	6
3. Resultat	7
3.1 Programmet	7
3.2 Egendesignade delar	12
3.3 Elkonstruktion	15
4. Diskussion	23
4.1 Programmet	23
4.2 Egendesignade delar	26
4.3 Elkonstruktion	30
5. Slutsatser	31
Tack	31
Referenser	32

1. Inledning

1.1 Bakgrund

Säkerheten i nya moderna bilar är markant högre än säkerheten hos omodernare bilar. Nyare bilar har även fler säkerhetsfunktioner och dessa hjälper till att minimera risker för olyckor. Exempel på dessa säkerhetsfunktioner är aktiv filhållningsassistans och system för att kunna avgöra avståndet till bilar framför sin egen. På grund av att olycksrisken är högre för omodernare bilar och att alla inte har de ekonomiska förutsättningar som krävs för att köpa en ny bil, finns ett uppenbart behov av att hitta alternativa säkerhetsfunktioner för äldre bilar.

PiVisions mål är att utveckla ett system för att fylla detta behov. Projektets viktigaste beståndsdel är en Raspberry Pi, vilket är en mikrodator som funktionellt sätt kan göra allt som en vanlig dator kan. I PiVision används den för att ta bilder. Dessa bilder omvandlas med hjälp av artificiell intelligens (AI) på ett kompatibelt AI-kort från Hailo på Raspberry Pi:n för att få fram avståndet till bilarna runt omkring bilen (Hailo, 2025a).

1.2 Syfte och frågeställningar

Syftet med detta gymnasiearbete är att undersöka möjligheterna att förbättra trafiksäkerheten hos bilar som saknar de vanliga säkerhetsfunktioner i nyare bilar på ett enkelt och hållbart sätt. Arbetet fokuserar på att analysera om en säkerhetsmodul, som exempelvis PiVision, kan bidra till att minska antalet trafikolyckor.

För att besvara detta undersöktes dessa frågeställningar:

- Behöver äldre bilar utan säkerhetssystem bytas ut?
- Hur påverkas trafiksäkerheten för äldre bilar vid installation av en säkerhetsmodul?
- Kan en säkerhetsmodul som PiVision minska antalet trafikolyckor?

2. Metod

2.1 Planering

Projektet började med att undersöka problemet. Detta gjordes genom att undersöka statistik på hur förarstödsystem minskar risken för olyckor. Enligt artikeln "Säker i bilen med förarstödsystem" (Folksam, u.å.) kan olycksrisken för bakändeskollisioner mer än halveras vid hastigheter upp till 50 km/h med hjälp av ett system som varnar för kollision och nödbromsar bilen vid behov. Denna statistik visar på att även bilar utan förarstödsystem kan bli säkrare av att implementera förarstödsystem. Därefter skapades en planering för att fördela den tillgängliga tiden för att säkerställa att projektets omfattning blev lagom.

2.2 Programmering

För att alla funktioner ska finnas krävs programmering som beskriver hur kameran ska ta foton, hur dessa ska behandlas, när bilen befinner sig i fara, med mera. Programmeringen ligger till grund för alla projektets system, förutom för hållaren, och beskriver hur de integrerar med varandra. All kod är skriven i programmeringsspråket Python, förutom hemsidan som utvecklades med hjälp av ChatGPT och Claude AI. Python användes då det är ett av de populäraste programmeringsspråken och har många användningsområden (Python, 2025). Detta språk kan också effektivt integreras och användas för att kontrollera en Raspberry Pi. Vidare fanns redan kunskap i gruppen om hur man programmerar i Python på grund av tidigare erfarenhet i detta programmeringsspråk.

Python innehåller flera olika funktioner som gör att den kan hantera data och göra olika saker som funktioner, loopar, klasser och så vidare. Språket har även en egen syntax som är som programmeringsspråkets grammatik. Till detta grundspråk finns även så kallade bibliotek, vilka är tillägg som kan installeras för att utöka funktionaliteten av språket.

Vid detta arbete har flera sådana bibliotek använts för alla funktioner. Ett exempel på ett bibliotek är picamera2 som kan användas för att programmera och använda kameror med Raspberry Pi. Utöver detta har supervision, som sköter hur bilderna hanteras genom datorseende (computer vision), använts. Vidare användes även biblioteket NumPy, vilket är ett system för att hantera listor. Dessutom användes threading som är ett bibliotek för att köra flera kodprogram simultant.

2.3 Elektronik

Elektroniken ligger även till grund för hur systemet fungerar. Elektroniken som användes erhöles genom att antingen beställa den online eller genom att återanvända delar som fanns tillgängligt. Ett av de första besluten som behövde fastställas var vilken sorts mikrodator som skulle användas till projektet. En mikrodator är som namnet säger en dator i mikroformat. En mikrodator är portabel, liten, multifunktionell och kompatibel med många andra elektriska komponenter.

Det finns två större alternativ i denna fråga, nämligen Raspberry Pi och Arduino. Frågan om vilken som skulle väljas var dock relativt enkel att lösa då båda dessa är ämnade att användas för olika saker (Newhaven Display, 2024). Arduino är passar bäst för att kontrollera elektronik och interagera med den. Den är enkel att använda i detta syfte och har många kompatibla elektroniska komponenter som kan användas med den. Å andra sidan har Raspberry Pi ett inbyggt operativsystem med starkare och större databehandlingsförmåga. Således lämpas sig Raspberry Pi för allt som kräver mycket bearbetning av data, vilket kan ses som ett av huvudbehoven i PiVision.

2.4 CAD

Alla egendesignade delar är skapade med hjälp av programmet Autodesk Inventor Professional 2021. Detta CAD-program användes eftersom störst erfarenhet fanns inom detta program i jämförelse med andra program. Dessutom var funktionerna som programmet erbjöd passande till projektets ändamål. Anledningen till att CAD användes för att designa komponenter är att det gav en god möjlighet att skapa prototyper genom 3D-utskrift. Med hjälp av dessa kunde problem med designen hittas och åtgärdas genom att justera felaktiga mått och ändra på designen.

3. Resultat

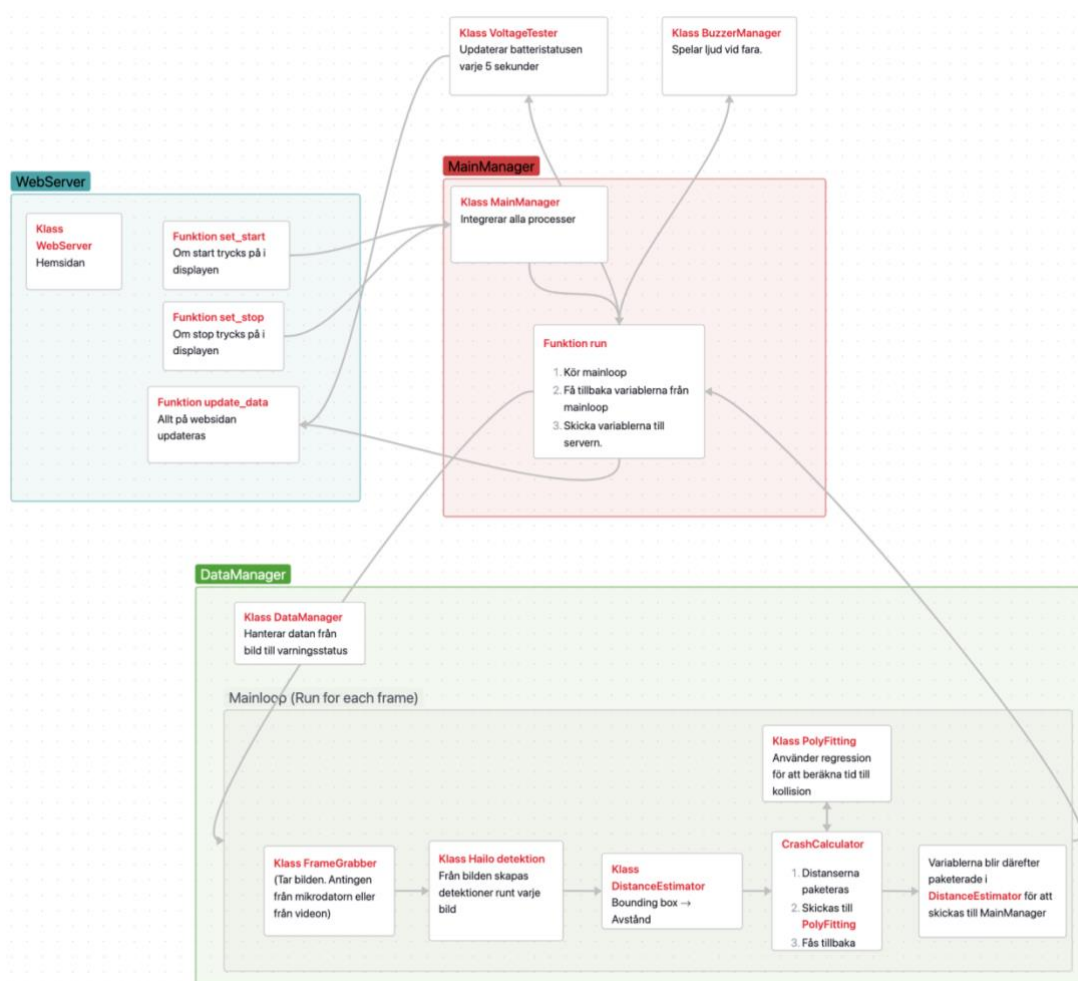
3.1 Programmet

3.1.1 Sammanhang

Programmet finns delat i kod på PiVision:s GitHub. (GitHub, 2025)

Programmeringen i PiVision består av flera olika kodprogram med olika klasser som har integrerats med varandra efter en struktur. I grunden finns klassen MainManager som sammanhåller alla andra klasser. En klass är som ett paket. Det sammanför variabler (olika värden) och innehåller olika funktioner. Dessa funktioner utför därefter olika handlingar. En funktion kan ses som en inkapslare av instruktioner, som gör att koden kan återanvändas flera gånger på flera platser. MainManager kan ses som hjärtat av programmet. I figuren nedan (Funktionsdiagram) visas de viktigaste klasserna och funktionerna.

WebServer, det vill säga en av de större klasserna, hanterar och kontrollerar hur hemsidan, det vill säga displayen ska fungera. Den andra större klassen DataManager hanterar datasystemet, där kollisionsstatus utvinns från bilden. Detta sker genom Mainloop, vilket är ett kodstycke som körs repeterande när programmet är aktivt.



Figur 1: Funktionsdiagram



Figur 2: Detektioner

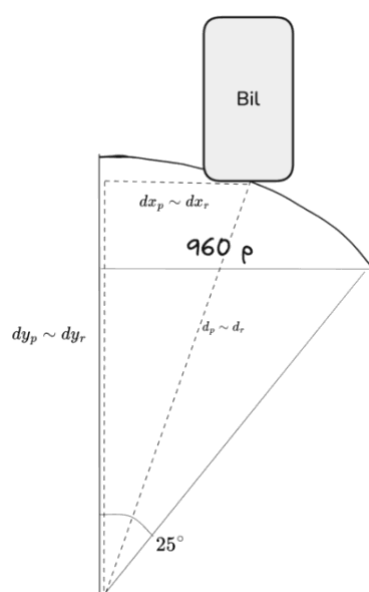
3.1.2 DataManager

Det första som DataManager gör i Mainloop är att skicka en förfrågan till klassen FrameGrabber. FrameGrabber tar därefter en bild och formaterar den. För att ta bilden importeras picamera2, ett bibliotek för kameraanvändning.

Bilden blir sedan anpassad genom en av de inre funktionerna. Denna anpassning ändrar bildens storlek utan att töja ut bilden. Således förbereds bilden för AI-bearbetning.

Nästa steg i processen är att skicka bilden till AI:n som processar bilderna i klassen Hailo Detection. Med detta gjort skapas boxar runt bilarna på bilden från mellan två XY-koordinater i pixlar (se figuren Detektioner). Därefter skickas boxarna över till subklassen DistanceEstimator.

3.1.3 DistanceEstimator



Figur 3: Estimeringsprocess

avståndet i sidled genom att använda ekvation 2. Slutligen användes Pythagoras sats i ekvation 3 för att få y-avståndet, alltså komponenten av avståndet rakt framåt.

$$\text{Avstånd framåt} \quad d_r = \frac{h_r}{h_p} \cdot F \quad (1)$$

Således användes denna ekvation. I ekvationen är h_r höjden i verkligheten på fordonet. Detta värde estimeras från en tabell. Alltså valdes det att använda ett värde för höjden på bil, ett för lastbil och ett för buss. h_p , det vill säga höjden i pixlar, fås å andra sidan genom att subtrahera pixelskillnaderna i y-led mellan hörnen på boxen, se Figur 4.

På formeln ovan utgör även d_r avståndet till bilen och F är kamerans brännvidd. Med detta menas det mått för på vilket avstånd kameran får fokus.

$$\text{Avstånd i x-led} \quad \frac{dx_r}{dx_p} = \frac{d_r}{d_p} \Rightarrow dx_r = \frac{d_r \cdot dx_p}{d_p} \quad (2)$$

Här utgör d_r avstånd rakt till bilen i verkligheten likt tidigare. Från den vill dess komponenter i x- och y-led erhållas. dx_r utgör avståndet i x-led, det vill säga sidledes i verkligheten, medan dx_p avståndet i x-led i pixlar.

För att dela upp d_r i komponenterna utnyttjades likformigheten mellan pixelavståndet och det riktiga avståndet i meter. Dessa kan tänkas vara två likformiga trianglar, se Figur 3. På grund av denna likformighet gäller att basen delat med höjden är lika för både triangeln i pixlar och i meter.

För att få ut dx_p krävdes två variabler. Först användes mitten av boxens avstånd till vänstra sidan av skärmen i pixlar. Därutöver krävdes även avståndet till vänstra kanten från mitten av skärmen i pixlar. När dessa subtraherades utvanns boxens avstånd med x-värdet för mitten av skärmen.

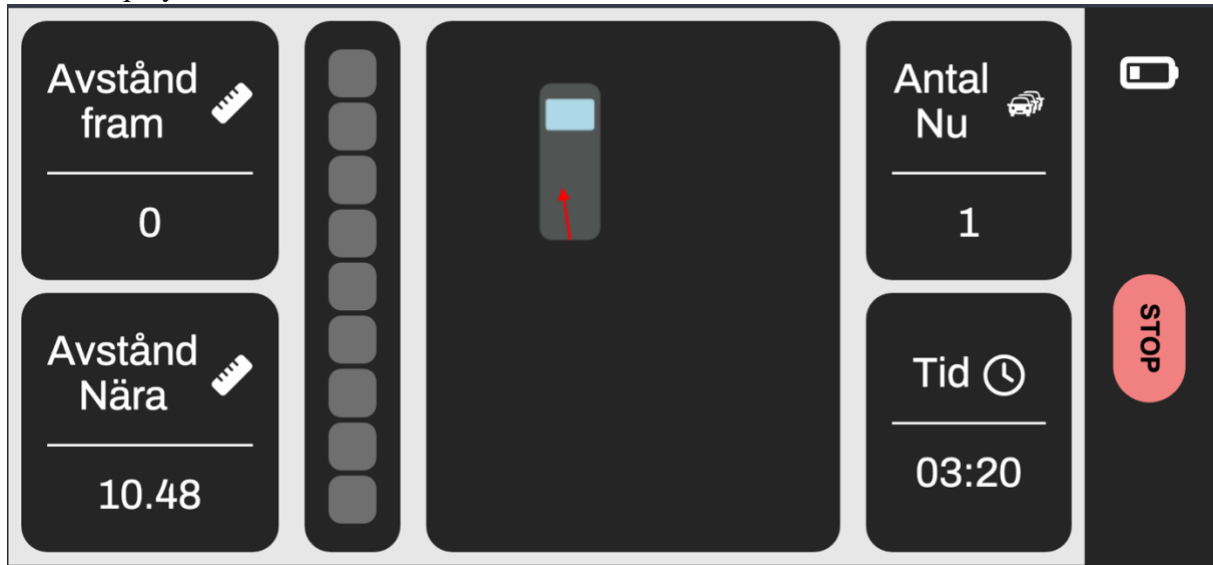
$$\text{Avstånd i y-led} \quad d_r^2 = dx_r^2 + dy_r^2 \Rightarrow dy_r = \sqrt{d_r^2 - dx_r^2} \quad (3)$$

Eftersom det nu finns två kända variabler kan Pythagoras sats användas för att räkna ut dy_r , det vill säga y-komponenten av d_r .

Dessa avstånd läggs sedan till i en begränsad lista som max kan innehålla 20 element. Om fler element läggs till kommer det äldsta elementet att raderas. Listan skickas därefter in i ett kontrolleringsprogram som beräknar tid till kollision. Processen sker genom att först skicka in de nuvarande distanserna med deras tillhörande tidpunkter, det vill säga när distansen mättes och vad avståndet var då. Detta sker för både avståndet i x- och y-led. Med hjälp av variablerna skapades sedan en viktad linjär regression. Viktningen gör att nyare värden blir prioriterade över äldre, vilket gör listans innehåll relevantare.

Med denna linjära funktion väljs 10 datapunkter ut mellan 1 och 3 sekunder in i framtiden. Om dessa datapunkter befinner sig inom arean av förarens bil både i y- och x-led skapar detta en varning. Denna varning varierar mellan grad 0 och 9 beroende på vilken av datapunkterna som befinner sig i bilens area. Dessa skickades slutligen tillbaka hela vägen till MainManager för att sedan visas på displayen.

3.1.4 Display



Figur 6: Exempel displayen

Likt varningssystemet hanteras displayen genom dess sammankopplas med MainManager. MainManager har subklassen WebServer (se funktionsdiagrammet), vilken hanterar websidan och uppdaterar den när ny information skickas in eller ut. Webbsidan körs på Raspberry Pi:n genom internetdelning och webbsidan kan därmed visas på en mobil.

Displayen är uppdelad i 7 paneler i rader och kolumner (Se Figur 6).

I första kolumnen finns två värden. "Avstånd Framåt" och "Avstånd Nära". Avstånd Framåt visar avståndet till bilen som befinner sig närmast framför bilen. Avstånd Nära visar i stället avståndet till bilen som är närmast.

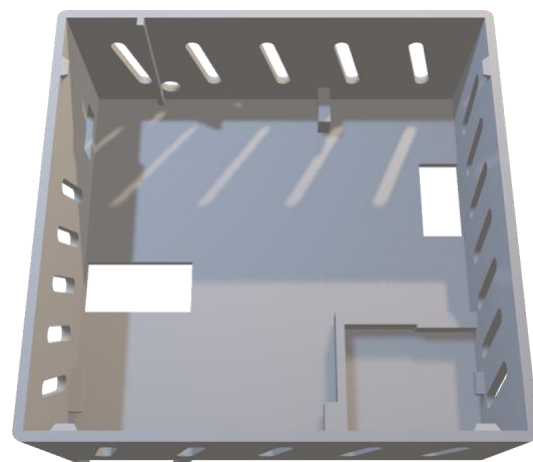
I andra kolumnen finns 9 kvadrater. Dessa fungerar något som ett trafikljus, i alla fall färgmässigt. I grad av att faran ökar tänds först de lägsta tre gröna kvadraterna en efter en, därefter tänds fler och fler tills alla är tända. När fjärde kvadraten tänds blir kvadraterna i stället orangea och när 7 eller fler är tända blir alla av kvadraterna röda. I tredje kolumnen finns slutligen en display som visar i realtid hur bilarna befinner sig och hur de antas röra sig.

I fjärde kolumnen visas "Antal Nu" och "Tid". Antal Nu beskriver hur många bilar som just nu hittas. Tid visar hur lång tid som har gått sedan systemet startades. I sista kolumnen eller sidofältet finns startknappen för att starta processen samt en ikon för batterinivån.

3.2 Egendesignade delar

Nedan visas de komponenter som 3D-modellerats och skrivits ut med hjälp av 3D-skrivare.

Hållare (se Komponent 1) som förvarar Raspberry Pi:n, kameran och de lösa komponenterna kopplade till Raspberry Pi:n.



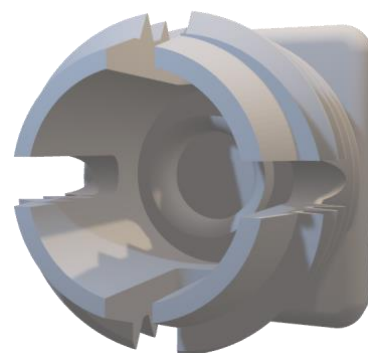
Komponent 1

Lock till hållare (se Komponent 2)



Komponent 2

Kulled (se Komponent 3) som fästs på hållaren för att kunna ansluta hållaren till ett vindruteffäste.



Komponent 3

Skruvgänga till kulle (se Komponent 4) som skruvas på kullen för att spänna åt den för att fästa vindrutefästet.



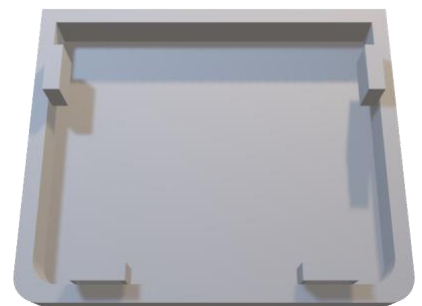
Komponent 4

Batterihållare (se Komponent 5) som limmas fast på Raspberry Pi-hållaren.



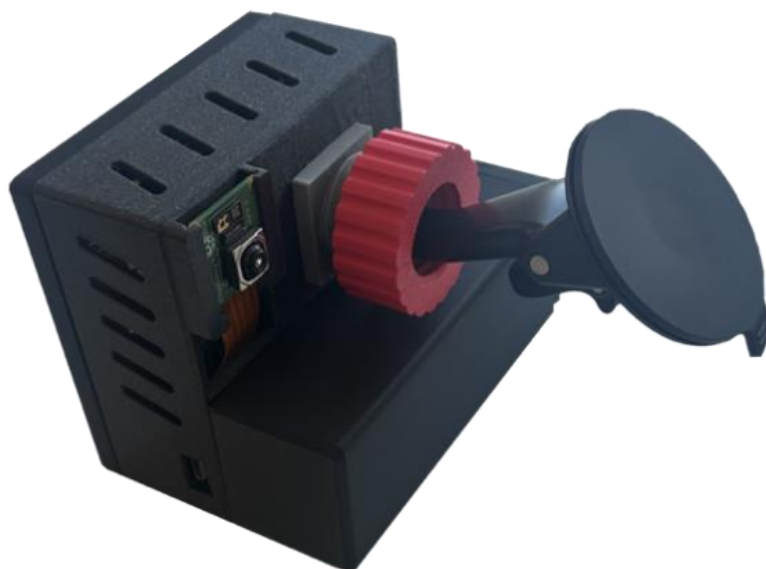
Komponent 5

Lock till batterihållare (se Komponent 6)

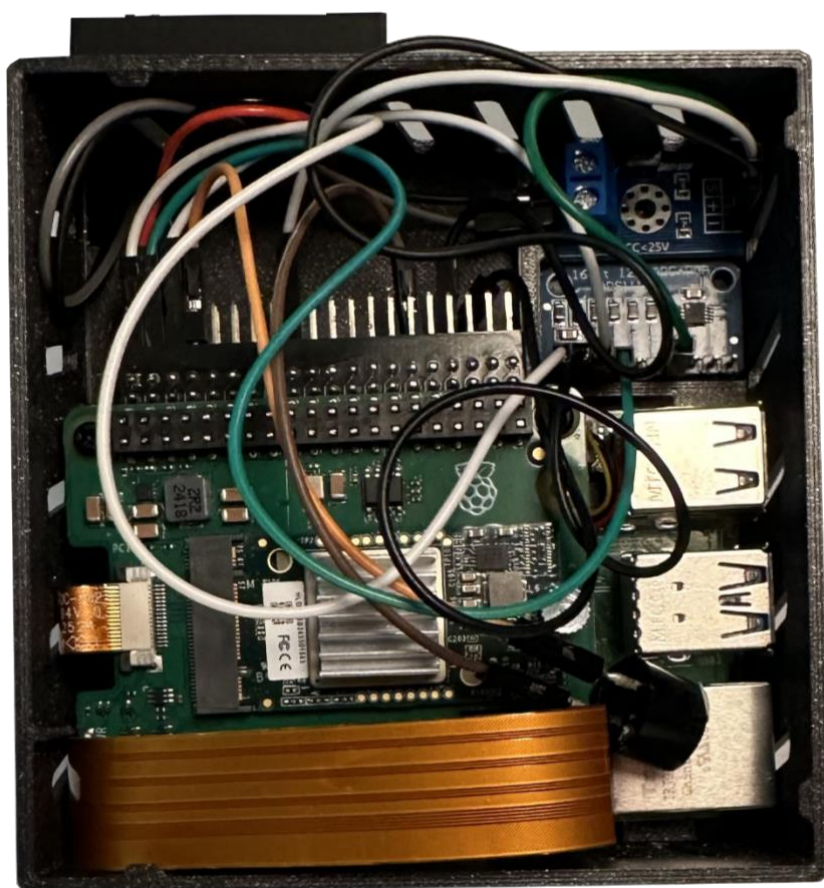


Komponent 6

3.2.1 Hållaren



Figur 7: PiVision komplett med alla delar

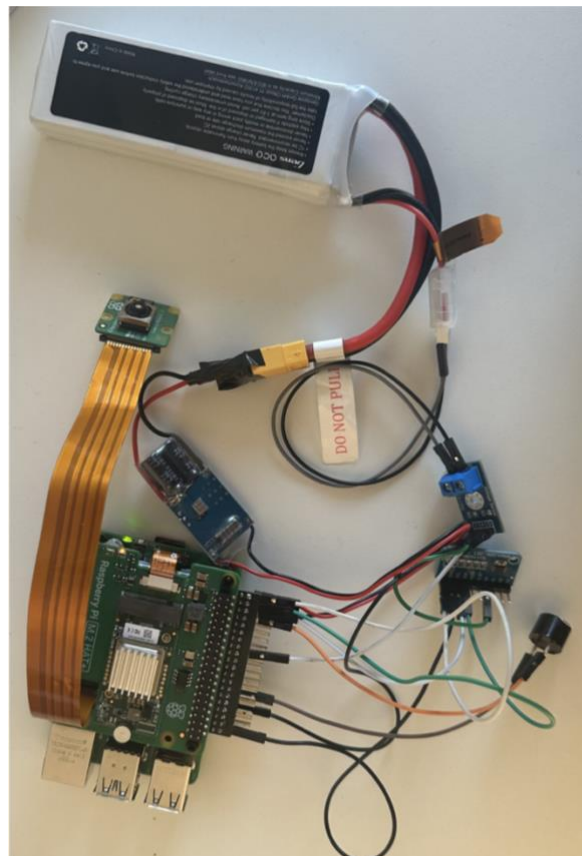


Figur 8: Hållaren framifrån utan lock

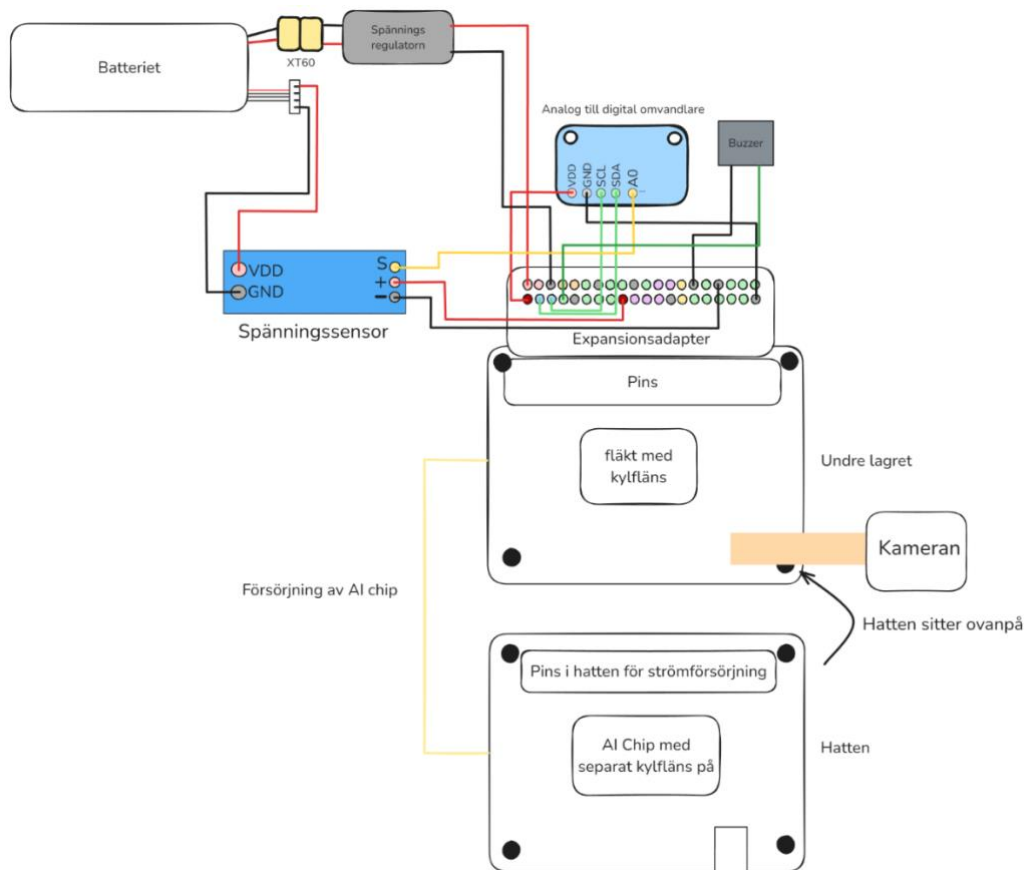


Figur 9: Hållaren framifrån med lock

3.3 Elkonstruktion



Figur 10: Elkonstruktion bild



Figur 11: Elkonstruktionsdiagram

Raspberry Pi2 GPIO Header

Pin#	NAME		NAME	Pin#
01	3.3v DC Power		DC Power 5v	02
03	GPIO02 (SDA1 , I ² C)		DC Power 5v	04
05	GPIO03 (SCL1 , I ² C)		Ground	06
07	GPIO04 (GPIO_GCLK)		(TXD0) GPIO14	08
09	Ground		(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)		(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)		Ground	14
15	GPIO22 (GPIO_GEN3)		(GPIO_GEN4) GPIO23	16
17	3.3v DC Power		(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)		Ground	20
21	GPIO09 (SPI_MISO)		(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)		(SPI_CE0_N) GPIO08	24
25	Ground		(SPI_CE1_N) GPIO07	26
27	ID_SD (I ² C ID EEPROM)		(I ² C ID EEPROM) ID_SC	28
29	GPIO05		Ground	30
31	GPIO06		GPIO12	32
33	GPIO13		Ground	34
35	GPIO19		GPIO16	36
37	GPIO26		GPIO20	38
39	Ground		GPIO21	40

Figur 12: Stiftens funktion (SparkFun, u.å.)

3.3.1 Allmänt

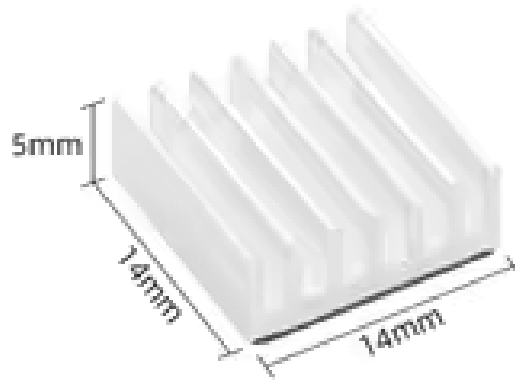
PiVision består av flera olika integrerade delar som tillsammans samarbetar genom elektriska kopplingar och programmering. I mitten av systemet finns Raspberry Pi:n som på dess ena sida har 40 anslutningsstift. Dessa stift är som små metallnålar som kopplingskablar kan anslutas till (Se Figur 10). Dessa kopplingskablar kopplas i sin tur till de olika externa komponenterna. Se elkonstruktionsdiagrammet för tydlig bild av alla kopplingar.

3.3.2 Fläkt och kylfläns

Den första komponenten närmast Raspberry Pi:n är dess fläkt och tillkommande kylfläns (metallobjekt som avleder värme). Fläkten och flänsen har som syfte att minska belastningen på systemet och bibehålla låg temperatur när systemet maximeras. Båda dessa sitter på översidan av Raspberry Pi:n och visas i Figur 13.



Figur 13: Fläkt och kylfläns



Figur 14: Separat kylfläns (AliExpress, u.å.-a)

3.3.3 Hatten och AI-kortet

För att kunna göra alla detektioner med AI:n och få tillräckligt snabba beräkningar krävdes ett AI-kort (se Figur 15, AI-kortet). Det skulle vara möjligt att enbart använda sig av Raspberry Pi:ns CPU (Computer Processing Unit), men detta skulle innebära mycket långsammare beräkningar. För att få plats och använda AI-kortet monterades det på en så kallad hatt (se Figur 16), det vill säga ett kretskort. Hatten monterades ovanpå Raspberry Pi:n med hjälp av gängade distansbussningar (De svarta pelarna i Figur 16). Dessa fästs sedan med skruvar på Raspberry Pi:n och på hatten. För att överföra den data och ström som krävs till hatten sammanlänkas den med anslutningsstiften samt en platt orange kabel (se Figur 16). För att undvika att AI-kortet skulle överhettas installerades en separat kylfläns direkt på AI-kortet (se Figur 14).



Figur 15: AI-kortet (Raspberry Pi, 2024)



Figur 16: AI-kortet på hatt på Raspberry Pi:n (Raspberry Pi, 2024)

3.3.4 Kameran

Kameran är ansluten med en lång kamerakabel som tillåter den att vridas och vändas för att få en bra vinkel (se Figur 17). Den fästs direkt på undre lagret (se Elkonstruktionsdiagrammet, Figur 11). Därefter sticker den upp ur hatten genom en öppning (se Figur 10).



Figur 17: Kameran (Electrokit, u.å.)

3.3.5 Expansionsadapter för anslutningsstiften

För att kunna använda anslutningsstiften till strömförsörjning och strömtestning när hatten är ansluten till alla stiften krävdes det att en expansionsadapter anslöts (se Expansionsadapter, Figur 18). På elkonstruktionsdiagrammet visas dessa pins eller stift med färgade cirkelar (se Stiftens funktion, Figur 12 och Figur 11).



Figur 18: Expansionsadapter (AliExpress, u.å.-b)

3.3.6 Strömförsörjningssystemet

För att systemet skulle kunna operera överhuvudtaget krävdes det en anslutningskabel och en strömkälla. Strömkällan som valdes är ett 3 cells 2200 mAh batteri från Gens Ace (se Figur 19). För att kunna ansluta detta batteri till Raspberry Pi:n krävdes en spänningsregulator eller en så kallad BEC (se Figur 20). Spänningsregulatorn tar in spänningen på 4.2 till 3.8 volt och varierande spänning från batteriet och omvandlar det till 5V och 5A som krävs för Raspberry Pi:n. Batteriet har två kontakter en balanskontakt för att se till att alla celler har samma spänning och en laddningskontakt varigenom batteriet används. Laddningskontakten på batteriet är den gula större kontakten, som heter XT60 (se Figur 21) hane-kontakten. Balanskontakten är den vita mindre kontakten som är instoppad till vänster på bilden. För att kunna koppla ihop spänningsregulatorn till XT60 honan på batteriet behövdes först en XT60 hane-kontakt lödas fast på spänningsregulatorn (Se Figur 19,20 och 21). Därefter kan XT60 honan och hanen sammankopplas för att koppla ihop spänningsregulatorn till batteriet.



Figur 19: Batteri (MBS-rcmodels, u.å.)



Figur 20: Spänningsregulator (AliExpress, u.å.-c)



Figur 21: XT60 kontakt Hane (Oskarshamns-Eskadern, u.å.)

3.3.7 Strömtestarsystemet

På själva displayen till systemet på mobilen ska batteriets nuvarande laddning kunna ses. För att möjliggöra detta användes en analog till digital omvandlare och spänningssensor (Se Figur 22 och Figur 23).

För att förbinda spänningssensorn till batteriet användes två kopplingskablar till batteriets balanskontakt. Eftersom spänningssensorn enbart ger ifrån sig analoga signaler krävdes även en analog till digital omvandlare som använde sig av I2C-gränssnittet för att se till att Raspberry Pi:n skulle kunna läsa informationen om spänningen. I2C är ett system utvecklat av Phillips Superconductors för att kunna dela information mellan komponenter. För att ansluta sensorn, omvandlaren och Raspberry Pi:n används ännu fler kopplingskablar.



Figur 22: Analog till digital omvandlare (AliExpress, u.å.-d)



Figur 23: Spänningssensor (AliExpress u.å.-e)

3.3.8 Summern

En ytterligare funktion som lagts till i projektet är att kunna använda en summer, det vill säga en liten högtalare, för att varna på ett tydligare sätt. Summern är kopplad till stiftet GPIO04 (Se Stiftens funktion, Figur 12). När programmet aktiverar GPIO04 kommer en signal att gå igenom summern och aktivera den för att varna föraren.

4. Diskussion

4.1 Programmet

4.1.1 Prestanda och Optimering

Är programmet tillräckligt snabbt och finns det några flaskhalsar? Från första början prioriterades att programmet skulle vara snabbt nog för att inte få prestandaproblem. För att säkerställa detta skapades hjälpklassen TimeTester som verktyg för att testa olika funktioner och se var största flaskhalsarna finns. Med hjälp av hjälpklassen kunde flaskhalsarna hittas och därefter programmeras bort. Således säkerställdes att programmet kördes effektivt. Vidare användes olika sorters visualisering för att kontrollera projektet. Till exempel användes biblioteket Rich för att visa antalet bilder per sekund som hanteras, vilket låg på runt 20 bilder per sekund (Rich, u.å). Rich erbjuder möjligheten att via terminalen visa information som uppdateras dynamiskt. 20 bilder per sekund ansågs även vara långt över det faktiska behovet, vilket gjorde att prestanda inte behövde prioriteras vidare. Om detta värde skulle minska drastiskt skulle TimeTester kunna användas för att leta reda på flaskhalsen för att sedan ta bort den. Tidsmässigt är även AI:n den största flaskhalsen i programmet, vilket gör att tiden för resterande kod kan försummas.

För att kunna nå ännu bättre resultat skulle till exempel AI-modellen kunna förbättras. Detta skulle kunna göras genom att till exempel ha ett mer kraftfullt AI-kort. Följden skulle bli snabbare resultat. Annars skulle även AI-modellen kunna minskas i storlek. Med en mindre AI-modell tar beräkningarna kortare tid att köras igenom. Dock är en nackdel med detta att det gör modellen mindre exakt. Å andra sidan kunde en annan AI valts som söker efter färre detaljer. AI:n som valdes har namnet YOLOv10n. YOLOv10n letar exempelvis efter tennisracket, zebbor, dattormöss, böcker och bilar i en lång lista på 80 detaljer när egentligen enbart 3 krävs för bil, buss och lastbil. Det skulle kunna ses som ett passande sätt att utveckla systemet. Vidare skulle det även leda till att AI:n skulle kunna skalas och bli bättre på att gissa rätt. Den största anledningen till att detta inte gjordes, utan att YOLOv10n valdes var att den var lättast att använda med AI-kortet.

I PiVision användes mycket av koden från Hailo:s Application Code, exempels som grund för hur AI:n skulle fungera med kortet och systemet (Hailo, 2025b). Om man däremot vill använda en annan AI genom att skapa den eller importera den, skulle det kräva tid utanför projektets ramar att färdigställa. En fördel med en egenskapad AI är en ökning i prestanda då den kan skräddarsys för PiVisions behov.

4.1.2 PiVision i olika situationer

En av de stora fördelarna med att använda just en AI för avståndsestimering är att den är väldigt funktionell i olika situationer. Användandet av AI tillåter även att kameran sitter på insidan av bilen och därför inte kan skadas av externa omständigheter som att vatten kommer in i systemet. Risken är också låg för att kameran ska påverkas av smuts, vatten och snö som hamnar på rutan.

Det negativa med användandet av ett AI-system är att man måste träna den på en stor datamängd. En till negativ faktor som skulle behöva testas för att kunna applicera PiVision i högre grad i verkligheten är hur väl den kan känna igen till exempel snötäckta bilar. Detta då snötäckta bilar inte är särskilt vanliga och att det finns en risk att AI:n inte känner igen dem som bilar på grund av bristen på träningsdata på dessa.

4.1.3 Avståndsberäkning från sidan

Vid det här laget fanns koden för hur bilden skulle tas och bounding boxen, det vill säga lådan runt bilen. Det som återstod var att försöka estimerar bilens avstånd som ett tre dimensionellt objekt som projiceras på en två dimensionell bild. Ett problem här var hur

avståndet till bilen skulle beräknas beroende på riktningen av bilen som upptäcks. Till exempel skiljer sig bredden av bilen markant om den har dörrarna mot kameran eller pekar mot förarens bil. Detta skulle vara en stor skillnad i bred på bounding boxen, utan någon faktisk avståndsskillnad. Den logiska lösningen var att i stället att enbart använda höjden av bilen. Oavsett om bilen nu står snett eller rakt kommer avståndet uppskattas ekvivalent. Å andra sidan sker det en förlust på information genom denna förändring. Från att kunna behandla två parametrar till att bara behandla en. En fortsättning på arbetet skulle kunna vara att direkt uppskatta avståndet med hjälp av AI eller rent av låta AI sköta även kalkyleringen av risk. På så sätt skulle systemet kunna förenklas avsevärt. Vidare leder det även till att problem som detta aldrig skulle ha existerat. I detta fall skulle dock en helt ny egengjord AI behövas.

4.1.4 Komposantuppdelning

Ett av de största problemen som uppstod under programmeringsprocessen var hur programmet skulle estimera fara om en bil åker förbi alldeles bredvid. Koden fungerade vid det laget så att bilden togs, bounding boxen skapades och avståndet estimerades. Detta problem skulle kunna uppstå om man till exempel kör nära bredvid en bil som står på vägkanten eller en bil kör förbi på andra sidan vägen. I så fall skulle programmet plötsligt mäta en bil som närmar sig i snabb fart och en stor ökning i bounding boxens höjd, vilket skulle leda till att systemet varnade i en faktisk ofarlig situation. Den egentliga data som då fanns var enbart bounding boxen, avståndet till bilen och vilken typ av fordon det var framför.

En lösning till detta problem skulle ha varit att använda en snävare del av bilden och försumma resten. Till exempel om bilen befann sig för mycket åt vänster eller höger skulle den räknas bort, men det skulle leda till att stora delar av all data skulle gå förlorad. Således bestämdes det att i stället dela upp avståndet i två komposanter, för att därefter avväga vad som faktiskt är fara.

Slutligen hittades ett sätt att skapa dessa komposanter genom att använda proportionaliteten mellan pixelavstånd och verkligt avstånd. För att kunna konvertera krävdes två värden som skulle vara proportionella med varandra. Dessa valdes att vara höjden av bilen i pixlar och dess verkliga höjd. För att få bilens höjd krävdes det dock att en approximation skulle göras. Varje bil fick en bestämd höjd på 1,5 meter och lastbil samt buss fick en höjd på 3 meter. Denna lösning skapar visserligen problem när bilarna inte har just den höjden, men eftersom det är förändringen av avstånd som är viktigast och höjden oftast ligger nära detta värde var lösningen mest passande. Därefter kunde proportionen mellan pixel och meter användas för att dela upp avståndet i dess komposanter. Med denna lösning kunde nu kollisionsrisken framåt och i sidled analyseras för sig, vilket löser problemet med bilar som åker förbi bredvid.

4.1.5 Utveckling av PiVision

Några andra lösningar som kunde ha valts var att genom AI ta in bilarna och från detta bestämma hur de var vinklade. En vinkel på 0° skulle betyda att den pekar mot oss och en vinkel på 180° ifrån oss. Denna lösning skulle lösa problemet med att försumma bilarna som åker mot oss och skulle även kunna användas för att tolka bredden på bilen på ett passande sätt. Å andra sidan skulle inte problemet som uppstår när man kör förbi en stillastående bil nära lösas och systemet skulle även förlora möjligheten att kalkylera risken att bilar från andra sidan av vägen också kan köra in i bilen. Denna lösning skulle även vara väldigt tidskrävande eftersom det skulle kräva ännu en objektigenkänningsmodell som även skulle behöva vara egengjord.

Ett problem som dock kan uppstå med denna metod av komponentuppdelning är hur faran bedöms när vägen är böjd. Det kan se ut som att bilen inte närmar sig särskilt mycket i sidledes även om det faktiska hotet för kollision är stort. Detta problem anses dock vara litet eftersom vägen måste svänga starkt för att den faktiska kollisionsrisken ska bli stor nog. Om vägen svänger enbart lite kommer modellen ändå se bilen som ett hot.

4.1.6 Parameteroptimering

Ett lätt sätt som programmet skulle kunna utvecklas är att försöka förbättra alla parametrar. Programmet har parametrar över hela projektet som möjliggör att programmet kan ändras utifrån behov. För att enkelt kunna korrigera parametrar och se till att alla parametrar finns lättillgängligt placerade i en klass med namnet "Parameters". Ett exempel på en viktig parameter är tröskelvärdet "score_threshold". Om "score_threshold" väljs till 40% innebär det att AI:n måste vara mer än 40% säker att bilen framför faktiskt är en bil för att den ska sparas som en upptäckt i systemet. Ett annat exempel på en parameter är "time_to_forget". Denna variabel behandlar hur lång tid det borde ta innan en bil ska glömmas helt från systemet. För att estimerar bilarnas avstånd måste man ha historik över tidigare positioner. Problemet med detta är att om för många av dessa sparas, kommer det leda till att minnet överflödas med gammal information som behandlas om och om igen. Följaktligen kommer avstånden att felberäknas.

Dessa parametrar skulle kunna försöka optimeras för att ge ett sådant bra system som möjligt. Om till exempel "score_threshold" är för lågt kommer objekt som inte är bilar att registreras som bilar. Vidare om den är för hög kommer inga bilar att upptäckas överhuvudtaget. För att lösa detta skulle parametrarna kunna testas i förhållande till andra variabler. Till exempel kan "score_threshold" optimeras för att antalet bilar som upptäcks ska bli så många som möjligt, samtidigt som antalet bilar som upptäcks fel blir så lågt som möjligt.

4.1.7 Större utveckling av systemet

Under tidens gång har flera olika val gjorts både för att begränsa eller utöka projektet. Till exempel valdes det från första början att använda AI, i stället för en använda lasersystem för att mäta avstånden. Detta val gjordes för att systemet skulle bli billigare, mer dynamiskt och inte påverkas lika mycket av externa faktorer som till exempel smuts på rutan, träd som bedöms som bilar osv. Ett begränsande val som gjordes var att enbart ha en kamera, när fler kameror kunde ha använts i olika riktningar. Detta skulle ha gjort systemet mer komplett i att finna faror, men även mer komplicerat.

Vidare skulle även systemet ha kunnat utvecklats för att finna fler faror. Till exempel skulle en detekteringsmodell ha kunnat använts för att se körfält och vita linjer. Om bilen åker över en vit linje utan att göra ett körfältsbyte skulle detta system kunna varna. Likväl skulle även personer kunna detekteras av systemet och varningen skulle kunna ta hänsyn till dessa. Om fler kameror hade installerats kunde även systemet fungerat som ett sätt att varna för till exempel fordon i döda vinkeln.

Här behövde ett aktivt val göras för att hålla programmet enkelt och inom tidsramen. I en framtida utökning av projektet kan var och en av dessa utökningar utforskas.

4.1.8 Skalbart program

Något som från första början eftersträvades var att få systemet att bli skalbart, det vill säga att det ska vara lätt att lägga till ny funktionalitet. För att säkerställa detta skapades all funktionalitet, som sagt, i paket med klasser.

Ett annat mål här var att se till att programmen kunde köras var för sig, vilket ledde till att felen lättare kunde isoleras. Till exempel om man enbart vill testa avståndet i DistanceEstimator kan enbart denna klass köras med testdata. Denna testdata samlades genom att använda PiVision och filma med den. Filmen kunde därefter återspelas för att verka som testdata.

Vidare användes som tidigare nämnt visualisering för att säkerställa att inga fel inträffar. En av de största anledningar till att buggar inträffar är ofta att någon av variablerna blir fel. Till exempel om h_p av någon anledning skulle bli 0 i ekvationen eller om Avstånd framåt i DistanceEstimator eller någon annan variabel skulle bli ett oväntat värde.

Likväl skulle ett fel kunna inträffa om Brännvidden F ges som en så kallad string (text) i stället för som en float (decimaltal). Det krävs därmed kontroll över vilka typer av variabler som hamnar var. Detta problem med fel typ kan ses som ett av de större problemen med dynamiskt typade programmeringsspråk. Följaktligen är ett av de största behoven att säkerställa att alla variabler som kan ges som olika typer har rätt typ från början.

4.1.9 Kostnad och Hållbarhet

Kostnaden för hela systemet just nu ligger på ungefär 2633 kr, vilket är en väldigt billig kostnad för att införa ett säkerhetssystem i en äldre bil. Dessutom skulle kostnaden minska stort vid större inköp av flertal komponenter samtidigt. Lösningen skulle kunna skräddarsys för att fungera väl och skulle inte behöva vara lika mångfunktionella som i vårt fall. Genom vår lösning krävs det inte ett inköp av en ny bil, vilket minskar användningen av både naturresurser och pengar. Produktionen av en ny bil släpper ut ungefär 11 till 14 ton koldioxid (Recurrent, 2025).

Pris per komponent:

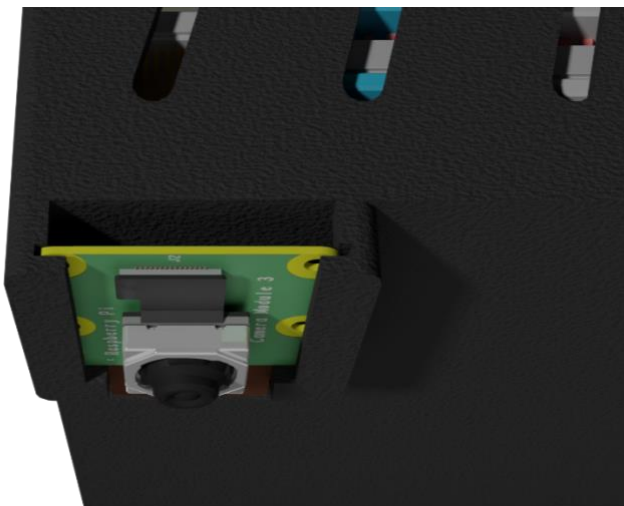
- Raspberry Pi 5: 864 kr
- Hailo AI-kort: 760 kr
- Minneskort 128 GB 160 kr
- Raspberry Pi kamera 300 kr
- Kamerakabel 24 kr
- Kylfläns 20 kr
- Spänningsregulator: 39 kr
- Anslutningsstift: 55 kr
- Analog till digital omvandlare: 35 kr
- Batteri: 260 kr
- Raspberry Pi fläkt: 100 kr
- Spänningssensor: 16 kr

4.2 Egendesignade delar

En av funktionerna hos den färdiga hållaren är att den erbjuder ett sätt att fästa Raspberry Pi-kameran för att ge en bra bild över trafiksituationen. Detta gjordes genom att skapa ett fäste till kameran på baksidan av hållaren, se Figur 24. Denna hållare fungerar på så sätt att kameran stoppas in ovanifrån i ett spår med ungefär samma tjocklek som kamerakomponentens kanter, se Figur 25. En annan funktion hos hållaren är att den försäkrar ett gott luftflöde till komponenterna genom lufthålen på hållarens sidor, vilket även visas i figur 24. Syftet med lufthålen är att undvika överhettning hos komponenterna. Dessa hål skulle även i framtida iterationer kunna optimeras för att optimera luftflödet ännu mer.



Figur 24



Figur 25

En ytterligare funktion hos hållaren är att den kan anslutas till ett redan existerande vindruteäste, se figur 26. Detta vindruteäste har ett klot längst ut och det är denna del som hållaren fäster i. Detta fungerar genom användandet av en kulle som sitter på hållaren, se komponent 3. Denna kulle fungerar på så sätt att klotet sätts in i kullen på hållaren och en kork skruvas sedan på för att spänna fast vindruteästet i kullen.



Figur 26: Vindruteäste

Det gjordes även möjligt att ta ur Raspberry Pi:n ur hållaren genom att använda på- och avtagbara lock, se Komponent 2. Detta lock fungerar på så sätt att delarna som sticker ut från hållarens väggar passar i urgröpningarna i Figur 27. En hållare till de lösa komponenterna kopplade till Raspberry Pi:n inuti hållaren skapades även. Detta visas i Figur 28. När hållaren designades låg även fokus på att minimera synligheten av sladdar för att öka estetikens skull. Detta gjordes till exempel genom att låta kamerasladden gå igenom hålet som visas i Figur 24.



Figur 27



Figur 28

4.2.1 Hållare

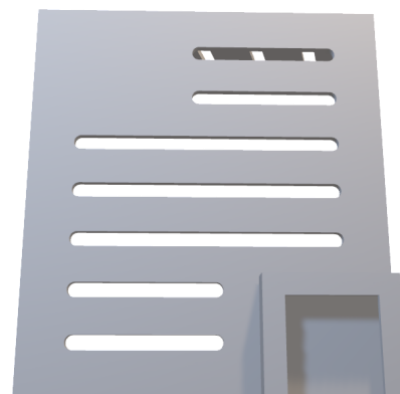
Ett fel som hittades med den första prototypen var att hållaren blev för smal för att få plats med de nya I/O pins som installerades på Raspberry Pi:n för att kunna ansluta externa elektriska komponenter. Detta löstes genom att göra den andra prototypen bredare. Då Raspberry Pi:n inte skulle kunna röra sig i sidled på grund av det större utrymmet, skapades två små väggar i den andra prototypen med ungefär samma bredd som Raspberry Pi:n.

I den första prototypen saknades det ett sätt att dra sladdarna till batterihållaren som skulle sitta till höger om Raspberry Pi-hållaren. Följaktligen skapades ett hål för sladdarna på den andra prototypens högra vägg (se Figur 31). Ett annat problem som uppstod var att det inte fanns något sätt att fästa två elektriska komponenter som var nödvändiga för att mäta spänningen. I den andra prototypen skapades därmed en hållare till dessa, se nedre högra hörnet i Figur 31.

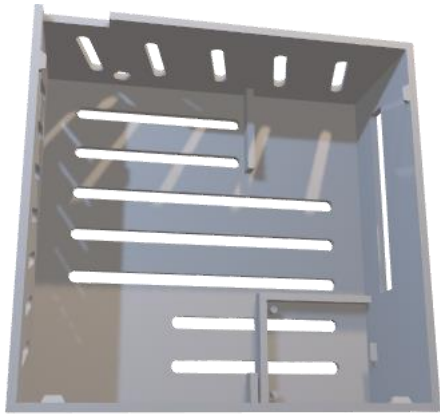
Kamerahållaren som visas i det nedre högra hörnet i Figur 30 fungerade inte heller då det inte fanns något utrymme för kamerasladden. I nästa prototyp skapades följaktligen ett hål med ungefär samma bredd som sladden på kamerahållaren, se Figur 32.



Figur 29: Första prototypen, framsida



Figur 30: Första prototypen, baksida



Figur 31: Andra prototypen, framsida

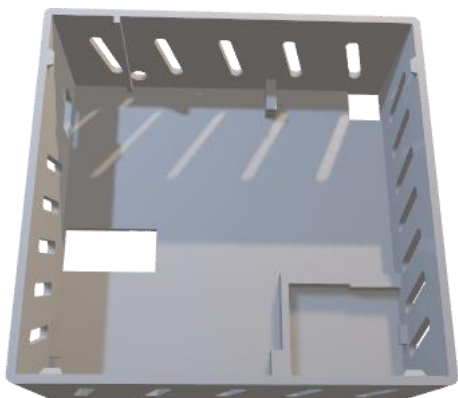


Figur 32: Baksida

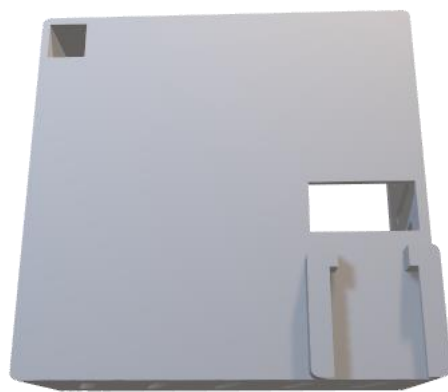
Den andra prototypen hade även problem som behövde åtgärdas. Till exempel fungerade inte hållaren i det nedre högra hörnet i Figur 31 och ändrades därmed till hållaren som visas i det nedre högra hörnet i Figur 33.

För att minska synligheten av kamerasladden togs utskärningen i det övre vänstra hörnet i Figur 31 bort och i stället designades ett hål för sladden som visas i Figur 34. Hela kameran och dess sladd träs in genom detta hål och kameran fästs sedan i hållaren. Den smala väggen i det övre vänstra hörnet i Figur 33 skapades för att hålla kamerasladden på plats. Utan denna riskerar sladden att blockera hålet till höger som används för att nå på och av knappen på Raspberry Pi:n.

Mellan prototyp två och tre ändrades även batterihållarens position. Innan prototyp 3 var tanken att fästa batterihållaren på sidan av Raspberry Pi-hållaren. Således designades hål i den högra väggen för att kunna dra sladdar igenom (se Figur 31). Detta ändrades i prototyp tre, där batterihållaren i stället fästs på undersidan av Raspberry Pi-hållaren. Därmed sattes hålet för sladdarna på botten av denna hållare. Lufthålen på botten togs även bort för att underlätta fästet av de komponenter som kommer sitta på undersidan, det vill säga, batterihållaren och kulleleden. Nu fanns även mer utrymme att limma fast komponenterna på. Anledningen till att lim användes var att en 3D-utskrift med allt material ihop skulle kräva för mycket material för att kunna testa olika prototyper. Detta då mycket stödmaterial skulle krävas. Ytterligare en förändring som gjordes var att kanterna rundades för att öka estetiken och för att göra hållaren mindre vass.



Figur 33: Tredje prototypen, framsida



Figur 34: Tredje prototypen, baksida

4.3 Elkonstruktion

4.3.1 Fastställandet av strömförsörjning

Ett problem i början av arbetet var hur Raspberry Pi:n skulle förses med spänning och ström. En Raspberry Pi kräver en spänning mellan 4,75 och 5,25 V samt en ström mellan 3 och 5 ampere. Här valdes 5 ampere som det mest passande, speciellt eftersom användandet av ett AI-kort leder till ett större strömanvändande.

I början fanns två större idéer. Antingen använda ett externt batteri eller använda det redan tillgängliga cigarettuttaget i bilen. Cigarettuttaget som idé är konceptuellt bra, men det fanns ett större problem, nämligen hur den skulle passa med rätt ström och spänning. Det skulle även krävas långa sladdar över insidan. Bilen som systemet främst skulle testas i hade även max 2 ampere i cigarettuttag. Således bestämdes det att använda ett externt batteri även om den har nackdelen att behöva laddas. Problemet med rätt ström och spänning fanns dock kvar och eftersom 5 V och 5 A är väldigt specifikt samt ovanligt för vanliga applikationer krävdes specialutrustning.

För att lösa problemet införskaffades en spänningsregulator för att konvertera till 5 V och ström upp till 40 A. Problemet med denna lösning var dock storleken på spänningsregulatorn. Detta berodde på att den i början var ämnad till en motor och därför i vårt fall innehöll "onödig" teknik för att även kunna reglera strömmen efter en motors behov. För att kunna minska storleken och även vikten av strömförsörjningsmodulen införskaffades en mindre spänningsregulator, vilket är den som användes.

4.3.2 Elkonstruktionen i praktisk användning och användarvänlighet

Något som skulle kunna förbättras är att göra lösningen mer användarvänlig. I nuläget krävs det att man vet precis var och hur alla delar ska sitta. Vidare krävs det att man vet hur man ska starta systemet och stänga av det på ett säkert sätt. Till exempel, måste man trycka på stänga av knappen två gånger och längre den andra gången för att stänga av Raspberry Pi:n. Först därefter bör man dra ur batterisladden. Ett enkelt sätt som systemet skulle kunna utvecklas, är att till exempel lägga till en knapp på displayen för att kunna stänga av Raspberry Pi:n i stället för att behöva klicka på knappen på Raspberry Pi:n.

5. Slutsatser

PiVision är ett innovativt mobilt förarstödssystem som syftar till att öka trafiksäkerheten för äldre fordon, genom att implementera moderna säkerhetsfunktioner som kollisionssvarning och avståndsmätning. Projektet har visat att det är möjligt att utveckla ett kostnadseffektivt och hållbart system, som kan monteras på befintliga fordon utan omfattande modifieringar.

Genom att använda en Raspberry Pi med AI-baserad bildbehandling kan PiVision analysera trafiksituationer i realtid, upptäcka närliggande fordon och beräkna avstånd för att varna föraren för potentiella kollisionsrisker.

Utvecklingsprocessen innefattade programmering i Python, design av 3D-komponenter med Autodesk Inventor och montering av elektroniska komponenter. Flera prototyper skapades för att hantera designutmaningar, och den slutliga produkten visar att systemet kan fungera effektivt i realtid med en prestanda på cirka 20 bilder per sekund. Detta gör PiVision till ett funktionellt verktyg för att minska olycksrisken hos bilar utan förarstödssystem.

Från resultaten kan slutsatsen dras att PiVision har potential att förbättra trafiksäkerheten på ett betydande sätt, särskilt för fordon som saknar moderna säkerhetsfunktioner. Vidare kan även slutsatsen dras att äldre bilar utan avancerade säkerhetssystem inte behöver bytas ut. Systemets modulära design möjliggör även för framtida utveckling och tillägg av nya funktioner, som körfältsdetektering eller varningar för fotgängare.

Sammanfattningsvis har PiVision demonstrerat att det är möjligt att integrera avancerad teknik i äldre fordon för att öka säkerheten och förlänga deras användbarhet. Projektet öppnar dörren för vidare forskning och utveckling inom området för kostnadseffektiva säkerhetslösningar för äldre fordon.

Tack

Vi vill rikta ett stort tack till Agustín Corbat för värdefull vägledning och stöd under arbetets gång. Vidare vill vi även tacka Carolina Wählby som förmedlade kontakten till Agustín.

Därutöver vill vi visa vår uppskattning till alla andra som hjälpt oss att framställa rapporten och gett stöd under projektets gång.

Referenser

AliExpress (u.å.-a). *Orange Pi Heatsink for CPU Passive Cooling 14x14 x6 MM Aluminum Heat Sink Cooler for OPI Zero 3 2 Banana Pi Raspberry Pi 4 3.*

[aliexpress.com/item/1005004447323577.html?spm=a2g0o.order_list.order_list_main.5.55941802lzWRaZ](https://www.aliexpress.com/item/1005004447323577.html?spm=a2g0o.order_list.order_list_main.5.55941802lzWRaZ). (Hämtad 24 mars 2025)

AliExpress (u.å.-b). *Raspberry Pi GPIO Header 1 to 2 Edge Extension HAT 40 Pin Expansion Adapter for RPI 4B 3B+ 3B Zero 2 Banana Pi Orange Pi.* [Länk till AliExpress](#). (Hämtad 24 mars 2025)

AliExpress (u.å.-c). *External UBEC 5V 3A / 5A / 7A / 15A BEC Multi-Purpose Step-down Power Supply Module Full Shielding Antijamming For FPV Airplane.*

https://www.aliexpress.com/item/1005003738477079.html?spm=a2g0o.order_list.order_list_main.10.55941802lzWRaZ. (Hämtad 24 mars 2025)

AliExpress (u.å.-d). *ADS1115 ADC Analog-to-Digital Converter Module with Programmable Gain Amplifier 16 Bit I2C 2.0V to 5.5V for Arduino Raspberry Pi.*

[.aliexpress.com/item/1005006917883058.html?spm=a2g0o.order_list.order_list_main.55.55941802lzWRaZ](https://www.aliexpress.com/item/1005006917883058.html?spm=a2g0o.order_list.order_list_main.55.55941802lzWRaZ). (Hämtad 24 mars 2025)

AliExpress (u.å.-e). *Voltage Detection Module Sensor For Arduino Electronic Building Blocks.*

[aliexpress.com/item/1005005647518766.html?spm=a2g0o.order_list.order_list_main.50.55941802lzWRaZ](https://www.aliexpress.com/item/1005005647518766.html?spm=a2g0o.order_list.order_list_main.50.55941802lzWRaZ). (Hämtad 24 mars 2025)

Electrokit (u.å.). *Raspberry Pi 5 Kamerakabel mini FPC 22-pin till FPC 15-pin 200mm.*

electrokit.com/raspberry-pi-kamerakabel-fpc-0.5mm-200mm (Hämtad 24 mars 2025)

Folksam (u.å.). *Säker i bilen med förarstödsystem.*

folksam.se/forsakringar/bilforsakring/forarstodssystem (Hämtad 10 mars 2025)

GitHub (2025). *PiVision*. github.com/GGisMee/PiVision (Hämtad 24 mars 2025)

Hailo (2025a). *The World's Best Edge AI Processors*. hailo.ai (Hämtad: 19 mars 2025).

Hailo (2025b). *Hailo Application Code Examples*. github.com/hailo-ai/Hailo-Application-Code-Examples (Hämtad: 18 mars 2025).

MBS-rcmodels (u.å.). *4S 2200mAh 30C Gens ace Soaring*. mbs-rcmodels.se/butik/lipo-4s-over-1000mah/gens-ace-4s-2200mah-30c-soaring (Hämtad 24 mars 2025)

Newhaven Display (4 september 2024). *Arduino vs Raspberry Pi: Key Features and Differences*. newhavendisplay.com/sv/blog/arduino-vs-raspberry-pi-key-features-and-differences (Hämtad: 19 mars 2025)

Oskarshamn-Eskadern (u.å.). *Kontakt XT60 Hane.*

http://www.oskarshamnsekskadern.se/shop/index.php?id_product=34&controller=product (Hämtad 24 mars 2025)

Picamera2 (u.å.). *picamera2 0.2.2*. pypi.org/project/picamera2/0.2.2. (Hämtad 3 april 2025)

Python (4 februari 2025). *Python (programming language)*.
[en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)). (Hämtad 3 april 2025)

Raspberry Pi (30 augusti 2024). *Raspberry Pi AI kit*. raspberrypi.com/products/ai-kit
(Hämtad: 19 mars 2025)

Recurrent (20 januari 2025). *Carbon Footprint Face-Off: A Full Picture of EVs vs. Gas Cars*.
<https://www.recurrentauto.com/research/just-how-dirty-is-your-ev>. (Hämtad 4 april 2025)

Rich (u.å.). *rich*. <https://github.com/Textualize/rich> (Hämtad: 27 mars 2025)

SparkFun (u.å.). *Raspberry gPlo*. learn.sparkfun.com/tutorials/raspberry-gpio/gpio-pinout.
(Hämtad 3 april 2025)